ATANDA COMMAND CENTRE · AUDIT RESPONSE · 26 OCT 2026

Gate Opening Runbook

One ordered sequence for clearing the gates in the 26 October production audit. Teal steps are prompts to paste into the Replit Agent. Amber steps are for a person, with a walkthrough for each. Work top to bottom: a step's "Needs" line says what must finish first.

`R-xx` Replit Agent prompt `H-xx` Human step `GATE` A gate decision only a named reviewer can make

Before you start: merge pull request #2

The fix (production guard install, verifier, runtime probe, serving-revision header) and this runbook are in ACC-FACTORY pull request #2. GitHub access (**H-01**) is already done. Review and merge the PR, then start with prompt **R-01**, option A.

STAGE 0

Get the fix into Replit

Nothing else can start until the code is in the Replit workspace.

`H-01` **Give Claude write access to ACC-FACTORY (or skip with the patch)**

Who: Repository owner **Time:** 5 min **Needs:** nothing

1. Open claude.ai/connect-github and reconnect your GitHub account.
2. Open github.com/apps/claude/installations/select_target, pick the `oluseye4233` account, choose **Only select repositories**, and tick `ACC-FACTORY` (and `ARK-X`, `atanda-fe` if you want them writable). Save.
3. Tell Claude "access fixed, push it". Claude pushes branch `claude/closed-gates-review-solutions-nzb79u`.
4. On GitHub, open a pull request from that branch into `main`, review it, and merge.

Skip option: download `closed-gates-c14e0fc.patch` from this Claude session, then upload it to the Replit workspace root (drag it into the Files pane). Use R-01 option B.

R-00 Standing rules (paste once at the start of every new Agent chat)

Needs: nothing

Replit Agent prompt

```
Standing rules for this whole conversation. Confirm you understand, then wait for the first task.

1. Never run DDL, migrations, "repairs", grants or data changes against PRODUCTION. Production schema changes only through Publish. The security installation in db/production-guard/ is run by a human administrator, never by you.
2. Never weaken, skip, broaden, mock out or quarantine a readiness check, policy comparison, trigger body, test, or security advisory to get a green result. A failed check stays failed; report it.
3. Development tests, mocks, synthetic receipts, configuration presence, green readiness panels and successful deployments are NOT production evidence or independent verification. Label every piece of evidence: DEV, SYNTHETIC, or PRODUCTION, with the date.
4. Tests must not make live model calls, send real email, contact real recipients or charge money. Use fixtures and @example.test users.
5. An uncertain paid model outcome must never trigger automatic redispatch or a duplicate charge.
6. Keep the Software and Firmware acceptance paths separate. Do not push software through hardware-only checks or remove firmware safeguards.
7. Do not mark anything ACTIVE in docs/upgrade-program/progress.json without dated production evidence. Use UNVERIFIED when unsure.
8. For each task: write a failing test first when it is a bug, make the smallest fix, run `pnpm -r typecheck` and the affected package tests, then commit with the task ID in the message.
9. Finish every task with a short report: files changed, commands run with pass/fail counts, and what is still unverified.
```

R-01 Bring in the audit fix and prove it in dev

Needs: H-01 **Closes:** setup for R1, R2, R3

Replit Agent prompt

```
Task R-01: bring in the production-audit fix (commit c14e0fc) and verify it in development only.

Option A (if main already contains it after the PR merge): run `git pull` and confirm `git log --
oneline -5` shows "Add reviewed production guard install, verifier and runtime probe; stamp serving
revision".
Option B (patch file): the file closed-gates-c14e0fc.patch is in the workspace root. Run `git am
closed-gates-c14e0fc.patch`. If it does not apply cleanly, stop and show me the conflict; do not
hand-edit the SQL files.

Then:
1. Make psql available (add pkgs.postgresql to replit.nix if `which psql` fails).
2. Run against the DEVELOPMENT database only:
   psql "$DATABASE_URL" -f db/production-guard/verify_artifact_operations_guard.sql
   Expect all 9 rows PASS, because migrations 0048/0050/0051 ran in dev. Paste the table.
3. Run the runtime probe against DEVELOPMENT:
   psql "$DATABASE_URL" -f db/production-guard/runtime_probe_artifact_operations.sql
   Expect PASS lines for role_switch, owner_isolation, cross_owner_insert_refused,
destructive_and_immutable_privileges_refused, and PASS or SKIPPED for final_immutable. It ends in
ROLLBACK.
4. Run `pnpm --filter @workspace/api-server test -- test/launch-readiness.test.ts`.
5. Do NOT run install_artifact_operations_guard.sql anywhere. Do NOT connect to production.

Done when: all 9 dev checks PASS, the probe passes, and the test passes. Any dev FAIL is a finding
to report, not something to "fix" by editing the checks.
```

STAGE 1 · AUDIT P0 · R1

Know exactly what is live

Match the published backend to a commit, and check what FORCE RLS will change before anyone installs it.

R-02 Make the serving-revision stamp work in the Publish build

Needs: R-01 **Closes:** R1 (code side)

Replit Agent prompt

```
Task R-02 (audit R1): make sure the published API reports the exact commit it was built from.

Context: artifacts/api-server/build.mjs now inlines the git commit as
process.env.ATANDA_BUILD_REVISION ("<sha>", "<sha>-dirty" or "unknown"). /api/health, /api/readyz
and /api/launch-readiness return it in the X-Serving-Revision header.

1. Find the build command the Publish/deployment config uses (.replit [deployment] build). Check
whether .git is available during that build. If it is not, change the deployment build command so it
sets ATANDA_BUILD_REVISION from the commit Replit is publishing. Never invent a value; "unknown" is
acceptable output, a guessed SHA is not.
2. Run the api-server build locally (`pnpm --filter @workspace/api-server run build`) with a clean
tree and confirm dist/index.mjs contains the current `git rev-parse HEAD` without "-dirty". Delete
dist afterwards if it is not tracked.
3. If anything during Publish writes tracked files before the build (which would make every build "-
dirty"), list it.
4. Add one line to docs/upgrade-program/closed-gates-opening-plan-2026-10-26.md under R1 saying how
the revision gets into the published build.

Do not publish. Report: the build command, whether .git is present at publish time, and the exact
header value you expect after Publish.
```

R-03 Find code that reads protected tables outside the restricted role

Needs: R-01 **Feeds:** H-04 security review

Replit Agent prompt

```
Task R-03: list everything that will behave differently in production once FORCE ROW LEVEL SECURITY
is installed. Read-only analysis plus tests; no schema changes.

Context: after the administrator runs db/production-guard/install_artifact_operations_guard.sql, the
five tables imprima_jobs, imprima_files, short_sprint_runs, artifact_operation_events and
sprint_verifier_authorities return NO rows to any connection unless app.operations_actor /
app.operations_admin are set, including the database owner. Inside withOperationsActor()
(artifacts/api-server/src/lib/artifact-operations/context.ts) this is already handled.

1. Search the whole monorepo (api-server routes and lib, cron jobs, scripts/, lib/db, exports,
janitors, analytics, admin pages) for every read or write of those five tables. Use the Drizzle
table objects (imprimaJobsTable, imprimaFilesTable, shortSprintRunsTable,
artifactOperationEventsTable, sprintVerifierAuthoritiesTable) and raw SQL table names.
2. Put each call site in a table: file:line, operation, inside withOperationsActor (yes/no), and
what happens under FORCE RLS (sees nothing / insert rejected / unaffected).
3. For each call site OUTSIDE withOperationsActor that would silently return empty or fail: write a
test that runs it against the dev DB (which already has FORCE RLS) and shows the behaviour. Do not
fix by adding BYPASSRLS, SECURITY DEFINER functions or owner fallbacks. Propose the fix (usually:
route it through withOperationsActor with the right actor), and wait for my approval before changing
code.
4. Save the table to docs/upgrade-program/force-rls-call-sites.md.

Done when: the document exists and every outside call site has a test showing its behaviour.
```

H-02 Publish and record the serving revision

Who: Release engineer **Time:** 15 min **Needs:** R-02, plus R-03 changes you approved

1. In Replit, open the **Publishing** tool and publish the current main (Autoscale, Public visibility, as today).
2. When it is live, run this in the Replit Shell or any terminal: `curl -sI https://command-center.atanda.ai/api/readyz | grep -i x-serving-revision`
3. Compare the value with `git rev-parse HEAD` for what you published. They must match exactly. A `-dirty` or `unknown` value means the build cannot be traced; go back to R-02.
4. Sign in to the live app and do one harmless round trip: open an existing session, reload, and check it is still there. Don't create paid or model-calling work.
5. Write the evidence record (template at the bottom of this page) with date, time, revision and result.

Evidence: header output, matching SHA, smoke-check result. **Clears:** audit R1.

STAGE 2 · AUDIT P0 · R2, R3

Install the production database guard

The only way to open the protected IMPRIMA / SHORT SPRINTS gate. Publishing again will never do it, because Publish does not carry roles, FORCE RLS, policy predicates or triggers.

H-03 Find out which database login the live app uses

Who: Administrator **Time:** 10 min **Needs:** H-02

1. In Replit, open the **Database** tool and switch to the **Production** database. Open its connection details. (If you also have the Neon console for this project, the same information is under **Roles** and **Connection details**.)
2. The production connection string looks like `postgresql://<USER>:<password>@<host>/<db>`. The part before the colon is the login. It is very likely `neondb_owner`; confirm it rather than assuming.
3. Check the deployment's secrets: if the published app has its own `DATABASE_URL` secret that differs from the Database tool's, the app's value wins. Use that login.
4. Write the login name down. You'll pass it as `app_login` in H-07. Never paste the password into chat, tickets or this document.

Evidence: “Published API connects as <login> (checked <date> via <where>).”

H-04 Security review of the install script

Who: Security reviewer (not the person who runs it) **Time:** 45–60 min **Needs:** R-01, R-03, H-03

1. Open `db/production-guard/install_artifact_operations_guard.sql` next to `lib/db/migrations/0048_awesome_marvel_zombies.sql` (lines 103–144), `0050_artifact_operations_integrity.sql` and `0051_artifact_operations_least_privilege.sql`. Every statement in the install should come from those three files. Mark anything that doesn't.
2. Check the privilege matrix in step 3 of the script against the table in `docs/upgrade-program/protected-workflow-readiness.md`. Nothing more than SELECT/INSERT/UPDATE as listed, no DELETE or TRUNCATE.
3. Confirm the role is created `NOLOGIN NOSUPERUSER NOBYPASSRLS` and the only membership granted is to the login from H-03.
4. Read `docs/upgrade-program/force-rls-call-sites.md` from R-03. Every call site outside `withOperationsActor` must either be fixed and published (H-02 again) or accepted in writing as “empty after install is correct”.
5. Confirm the script stops on its own if it finds an unreviewed permissive policy, and rolls back if the final nine-check self-test fails.
6. Approve in writing (ticket or email) with your name, date and the commit SHA you reviewed.

Evidence: signed approval naming the reviewed SHA. **Rule:** if the reviewer wants the SQL changed, it goes back through R-xx as a code change and a new review, not an edit on the server.

H-05 Take a restore point and run the read-only pre-check

Who: Administrator **Time:** 15 min **Needs:** H-04

1. **Restore point.** If you have the Neon console: **Branches → Create branch**, parent `main`, "current point in time", name it `pre-guard-install-YYYYMMDD`. Otherwise use Replit's database backup/restore option for Production and note the restore timestamp. Don't continue without one.
2. **Get psql in the Replit Shell:** `nix-shell -p postgresql` (or after R-01, psql is already installed).
3. **Put the production admin connection string in a shell variable** without echoing it: `read -rs PROD_ADMIN_URL; export PROD_ADMIN_URL`, then paste and press Enter.
4. **Pre-check (read-only):**
   `psql "$PROD_ADMIN_URL" -f db/production-guard/verify_artifact_operations_guard.sql`
5. You should see the same failures as the audit: `restricted_role`, `forced_rls`, `owner_policies`, `authority_policies` and `immutable_versions` FAIL, three FAIL PREREQUISITE, and `policy_scope` PASS. Save the output. If `policy_scope` FAILs, stop: someone added a policy since the audit and it needs review first.

H-06 Run the installation (maintenance window)

Who: Administrator, with the reviewer present **Time:** 10 min, in a quiet period **Needs:** H-05

1. Announce a short window. The script takes table locks for a few seconds (it gives up after 10 s rather than queueing behind traffic).
2. From the repository root, run (replace the login with the value from H-03): `psql "$PROD_ADMIN_URL" -v app_login='neondb_owner' -f db/production-guard/install_artifact_operations_guard.sql`
3. **Success** looks like a table of nine rows all `PASS`, then “Installed. Now run the actual-runtime-role probe…”.
4. **Failure** prints `ERROR: Guard self-check failed, rolling back: <check>` or another error. Nothing was changed. Save the output and send it back as a code task. Never edit the script on the server to get past it.
5. If it fails with a lock timeout, wait for a quieter moment and run it again; it is safe to repeat.

Rollback after a successful commit: restore the H-05 restore point. Don't drop individual guards by hand; that leaves a lane that looks protected but isn't.

H-07 Prove it with the app's own identity

Who: Engineering + reviewer **Time:** 20 min **Needs:** H-06 **Clears:** R2, R3

1. Put the **published app's** connection string (from H-03) in a variable: `read -rs API_DATABASE_URL; export API_DATABASE_URL`.
2. Run `psql "$API_DATABASE_URL" -f db/production-guard/runtime_probe_artifact_operations.sql`.
3. Expect NOTICE lines: `PASS role_switch`, `PASS owner_isolation`, `PASS cross_owner_insert_refused`, `PASS destructive_and_immutable_privileges_refused`, and `PASS final_immutable` or `SKIPPED final_immutable` if no Published Final document exists yet. It ends with “transaction rolled back”, so nothing was written.
4. Run the verifier again with the same URL: `psql "$API_DATABASE_URL" -f db/production-guard/verify_artifact_operations_guard.sql`. All nine must PASS, and the membership row should name the app's login.
5. Sign in to the live app as an administrator and open the protected-workflow verifier panel. It must show the database guard as ready. Take a screenshot.
6. Save all outputs in one dated evidence record with the serving revision from H-02.

GATE H-08

Decide on the protected-database gate

Who: Administrator + security reviewer + engineering lead **Needs:** H-07

- Only **“database guards: PASS on production under the runtime identity”** can be marked here. This is not independent verification, external approval or a release decision for IMPRIMA / SHORT SPRINTS.
- Update the gate register: Protected IMPRIMA and SHORT SPRINTS moves from CLOSED to “guards installed; pilots pending (R8)”.
- Keep a monthly re-run of the read-only verifier on the calendar. Any later FAIL closes the gate again.

STAGE 3 · AUDIT P0/P1/P2 · R4, R5, R6, R12

Core reliability and honest status

Can run alongside Stage 2. Each regression gets its own prompt so the Agent works on one thing at a time.

R-04

Evidence fields for the invariant register

Needs: R-02 **Closes:** R4 (tooling)

Replit Agent prompt

```
Task R-04 (audit R4): make docs/upgrade-program/progress.json able to hold real, dated evidence per invariant, without changing any entry's status.

Context: all 129 invariants and 243 atomic prompts are liveActivity: UNVERIFIED. The audit says to track implementation, live activity and verification separately and never turn 0/129 into a whole-platform percentage.

1. Extend the schema of each entry (keep existing fields) with: owner, implementationState (MISSING|PARTIAL|IMPLEMENTED), devVerification {type, result, date, synthetic: true/false}, liveActivity (ACTIVE|INACTIVE|UNVERIFIED), liveEvidence {servingRevision, date, assessor, enforcementTier, link}, blockingDependency, exitCondition.
2. Write scripts/src/invariant-register.ts with two commands:
   - `check`: fails if any entry is ACTIVE without liveEvidence.servingRevision + date + assessor, or if devVerification.synthetic is true while liveActivity is ACTIVE.
   - `summary`: prints counts per state. Never prints a single completion percentage.
3. Add `check` to the existing scripts test command.
4. Do NOT change any entry's liveActivity. Fill owner/implementationState only where the source makes it obvious; otherwise leave them blank.

Done when: the checker runs in tests, all entries are still UNVERIFIED, and the summary output is in your report.
```

R-05 Regression #330: public certificate check makes staff-only calls

Needs: R-00 **Closes:** part of R5

Replit Agent prompt

```
Task R-05 (audit R5, task #330): the public certificate verification page must not call staff-only endpoints.

1. Load the public certificate verification page signed out (Playwright, dev server). Record every network request it makes. Any request to an endpoint that requires staff/admin auth, or that returns 401/403 and shows an error, is the bug.
2. Write a failing test that asserts the signed-out page makes only public requests and shows no auth error.
3. Fix it with the smallest change (usually: a public endpoint returning only the fields verification needs, or not calling the staff endpoint when signed out). Do not loosen auth on the staff endpoint.
4. Run the test and the existing certificate tests.

Report: the requests before and after, files changed, test results.
```

R-06 Regressions #112 and #239: stage unlocking and F0 foreign artifacts

Needs: R-05 **Closes:** part of R5

Replit Agent prompt

```
Task R-06 (audit R5, tasks #112 and #239). Do them one at a time, committing after each.

#112 stage unlocking:
1. Find where F-stages unlock (the code that decides a user may open the next stage). Write tests for: a stage must not unlock before its prerequisite passes; a failed or pending gate keeps it locked; reloading does not unlock it; a different user's progress never unlocks mine.
2. If a test fails, fix the server-side check. Do not rely on hiding a button in the UI.

#239 F0 foreign artifacts:
1. Write a test where user A tries to attach, view or monitor an artifact owned by user B through every F0 endpoint (retainer create, monitoring run, alerts).
2. Each must be rejected with 404 or 403 and leave no row behind. Fix any that accept it, by checking ownership in the server.

Report per issue: reproduced yes/no, root cause, fix, tests.
```

R-07 Regressions #93 and #195: free-tool score exposure and F9 duplicate retries

Needs: R-06 **Closes:** part of R5

Replit Agent prompt

```
Task R-07 (audit R5, tasks #93 and #195). One at a time, commit after each.

#93 free-tool score exposure:
1. List every response the free (unauthenticated or free-tier) tools return. Write a test asserting
they never include scores, rubric internals, other users' data or paid-tier fields that the product
says are gated.
2. Fix the response shape on the server. Do not just hide fields in the UI.

#195 F9 retry duplicate protection:
1. Write a test that fires the same F9 retry twice concurrently and twice in sequence with the same
idempotency key.
2. Assert exactly one model dispatch, one cost row and one result. An uncertain earlier outcome
(timeout, unknown) must be marked for operator review, not redispatched automatically.
3. Fix with a database-level unique key or row lock, not an in-memory flag.

Report per issue: reproduced yes/no, root cause, fix, tests. No live model calls; mock the provider.
```

R-08 Regressions #92 and #296: rate-limit restarts and budget drift

Needs: R-07 **Closes:** rest of R5, part of Priority 2

Replit Agent prompt

```
Task R-08 (audit R5, tasks #92 and #296, plus Priority 2 budget accounting).

#92 rate-limit restart behaviour:
1. Find where rate limits and daily caps are stored. If any limit lives only in process memory,
write a test showing that a server restart resets it (Autoscale restarts often).
2. Move the counter to the database (or confirm it is already there) so a restart does not reset
limits.

#296 budget drift and model-call accounting:
1. Write tests for the three outcomes of a paid model call: completed, failed, and uncertain
(timeout or connection lost after sending).
   - completed: cost recorded once.
   - failed before sending: no cost.
   - uncertain: cost recorded as "reserved/uncertain", visible to operators, NOT redispatched
automatically, and reconciled later.
2. Write a test that the company-wide monthly cost cap and the per-user budget agree with the sum of
cost rows (no drift after retries, cancellations or crashes mid-call).
3. Fix what fails. Keep the existing cap behaviour: halt spend when reached.

Report: tests added, failures found, fixes. Mock all providers.
```

R-09 Recovery alerts and operator visibility

Needs: R-08 **Closes:** Priority 2; tasks #327, #328, #331, #332, #187, #230

Replit Agent prompt

```
Task R-09 (audit Priority 2; tasks #327, #328, #331, #332, #187, #230). Work in this order and
commit after each:

1. #327 recovery while the API sleeps: Autoscale can scale to zero. Make sure abandoned credits and
interrupted private files are found on the next wake-up or cron tick, not only by an in-process
timer. Test by simulating a restart mid-operation.
2. #328 repeated recovery-failure alerts: if recovery of the same item fails repeatedly, alert
ADMIN_EMAILS once per episode (reuse the pattern in cron_stale_notifications), not on every attempt.
3. #331 individual alert inspection: an admin can open one alert and see its item, attempts, last
error and owner.
4. #332 bounded recovery audit history: keep recovery history per item capped (like
cron_tick_events, newest N rows), with a test for the pruning.
5. #187 cross-tab Player retry status: a retry started in one tab shows as running in another tab
for the same user, without starting a second retry.
6. #230 paused-alert retry performance: retrying paused alerts must not scan the whole table; add an
index or bounded query and a test with many rows.

No real emails in tests (EMAIL_FROM unset gives dry-run logging). Report per item.
```

R-10 Webhook failures, monthly provider bill and consistent statuses

Needs: R-09 **Closes:** Priority 2; proposals #188, #302

Replit Agent prompt

```
Task R-10 (audit Priority 2; proposals #188 and #302). These were PROPOSED, not accepted: first
check whether they are already done, and report before building.

1. #188 webhook delivery failures: when an outbound webhook fails repeatedly, show a warning to the
owner and admins, with retry history. Test with a fake endpoint that returns 500.
2. #302 monthly provider-bill reconciliation: an admin export (CSV) per month of model cost rows
grouped by provider and model, with totals, so it can be compared with the provider's invoice. Read-
only; no changes to cost rows.
3. Consistent session statuses and engine names: list every place the UI shows a session status or
engine name, and make them use one shared mapping. Add a test that fails if a new status has no
label.

Report what already existed, what you built, and tests.
```

H-09 Create test accounts for signed-in journeys

Who: Product owner **Time:** 20 min **Needs:** nothing

1. In the **Clerk dashboard**, development instance, create one user per role you sell: free, Practitioner, Architect, team convener, team trainee, admin. Use addresses on `@example.test` so the cleanup janitor treats them as test data.
2. Store their passwords as Replit Secrets (for example `E2E_PRACTITIONER_PASSWORD`) in the **development** environment only. Don't create these users in production.
3. Tell the Agent the secret names before R-11.

R-11 Signed-in core journeys, end to end

Needs: H-09 **Closes:** R12 (dev side); tasks #323, #324, #325, #333, #334, #344

Replit Agent prompt

```
Task R-11 (audit R12): Playwright journeys for the signed-in core, against the DEV app, using the
test accounts in secrets E2E_* (names: <fill in from H-09>).

For each journey: sign in, do the work, reload, sign out and in again, and check the work is still
there. Then sign in as a DIFFERENT user and check it cannot see or change it (negative owner test).
1. Quest: start, save, reopen (#334 Quest evidence accessibility too: keyboard-only and screen-
reader labels).
2. Cohort: convener invites, trainee joins, convener sees progress (#344 convener/trainee signed-in
checks; #323 invited lesson live check).
3. SPC Player: pick a source, run, and confirm the source file is kept for the real run (#325).
4. Boomerang: return a candidate, rework to Upgraded 1, then Upgraded 2; reload persists; stale or
foreign live check is refused (#324).
5. Preparation and archive: create, archive, restore.
6. #333 signed-in supporting-work recovery: interrupt mid-save (kill the request) and recover.

Rules: mock all model calls; no real emails; clean up only @example.test users. Commit the specs
under e2e/ and add a `pnpm e2e:core` script.
Report: pass/fail per journey with screenshots of failures. A failure is a finding; do not change
the test to pass.
```

R-12 Accessibility, mobile and Boomerang follow-ups

Needs: R-11 **Closes:** Priority 3; tasks #315, #316, #329, #321, #322, #348, #349

Replit Agent prompt

```
Task R-12 (audit Priority 3). One commit per item.

1. #315/#316 high-contrast screens and text/contrast: run axe (via @axe-core/playwright) on every
main page at 390px and 1280px widths, light and dark. Fix contrast and missing labels. Add the axe
run to the e2e suite so it fails on new violations.
2. Keyboard: every action on Quest, Player, Boomerang and cohort pages works with Tab/Enter/Escape,
with visible focus.
3. #329 F-stage bundle loading on mobile: measure the JS loaded on first visit to each F-stage page;
split anything over 250 KB gzipped that the page doesn't need first.
4. #321 stale Boomerang work warnings: warn when a rework is based on a source that has since
changed.
5. #322 review-history search.
6. #348 non-F8 revision comparison: side-by-side diff for non-F8 Boomerang revisions, before
acceptance.
7. #349 Boomerang/gate-history export: CSV/JSON export of upgrades and gate decisions for one
session.
Rules: no inherited pass scores or certification on any new version (existing Boomerang rule).
Report per item.
```

R-13 Paid journey: code side

Needs: R-11 **Closes:** R6 (code side); tasks #346, #347

Replit Agent prompt

```
Task R-13 (audit R6; tasks #346, #347): make sure what we show, what Stripe charges and what the
user gets all match. Stripe TEST mode only.

1. List every plan and price shown in the UI (pricing page, upgrade dialogs) and every
STRIPE_PRICE_* variable the server uses (PRACTITIONER_MONTHLY/YEARLY, ARCHITECT_MONTHLY/YEARLY,
TEAM_SEAT_*, TEAM_LITE_SEAT_*, INGESTION_PROJECT, CARTRIDGE_PROJECT). Write a test that fails if a
displayed plan has no configured price or vice versa.
2. Write a script `pnpm --filter @workspace/scripts run stripe-price-check` that fetches each
configured price from Stripe (read-only) and prints amount, currency, interval and active, so a
human can compare it with the pricing page. It must never create or change prices.
3. #347 entitled workspace entry: test that after a successful checkout.session.completed webhook
(use Stripe test fixtures, signed with STRIPE_WEBHOOK_SECRET), the buyer gets exactly the
entitlement bought and can enter the paid workspace; a cancelled or failed payment gives nothing;
replaying the webhook does not double-grant.
4. Test account ownership and recovery: a paid user who deletes and re-creates sessions keeps the
subscription; another user can't take it over.
5. Do not enable checkout in production or change its current disabled/dormant state.

Report results and the price-check output from TEST mode.
```

H-10 Check prices in Stripe and do one real purchase

Who: Business owner **Time:** 45 min **Needs:** R-13 **Clears:** R6

1. Decide the final price list. In the **Stripe dashboard**, switch to **Live mode**, open **Product catalog**, and create or confirm one Product per plan, with a Price for each monthly/yearly option. Copy each Price ID (starts with `price_`).
2. In Replit, open the deployment's **Secrets** and set the matching `STRIPE_PRICE_*` values, plus the live `STRIPE_WEBHOOK_SECRET`.
3. In Stripe, **Developers → Webhooks → Add endpoint**: URL `https://command-center.atanda.ai/` plus the webhook path the Agent reports in R-13, with the events it lists. Copy the signing secret into `STRIPE_WEBHOOK_SECRET`.
4. Run the price-check script from R-13 against live (read-only) and compare it line by line with the pricing page.
5. Turn checkout on only when you're ready. Then buy the cheapest plan with your own card, confirm you land in the paid workspace, and refund yourself in Stripe (**Payments → the payment → Refund**).
6. Record the date, plan, Stripe payment ID and result.

R-14 Dependency advisories: try to remove the sprintf-js path

Needs: R-00 **Closes:** Priority 4 (partly)

Replit Agent prompt

```
Task R-14 (audit Priority 4). Follow docs/upgrade-program/dependency-advisories.md.

1. Re-run `pnpm audit --json` and record today's result and the latest registry versions of braces,
sprintf-js, micromatch, http-proxy-middleware, mammoth and argparse.
2. sprintf-js comes in through mammoth → argparse@1 → sprintf-js. argparse@2 has no sprintf-js
dependency, but its API differs, and Mammoth only uses argparse in its command-line tool. Try a
parent-scoped pnpm override `mammoth>argparse: ^2.0.1` (same style as the existing typography
override). Then:
   - run `node --test scripts/src/dependency-boundaries.test.mjs` and the DOCX ingestion tests;
   - extract text from a real DOCX through the app's library path (extractRawText);
   - confirm pnpm audit no longer lists sprintf-js.
   If anything fails, revert the override and report why. Do not use Mammoth's CLI.
3. braces: if there's still no patched release, leave it open. Do not add an audit suppression. Re-
run the existing reachability test and record the date.
4. Update dependency-advisories.md with a dated "Recheck" section.

Report: audit before/after, tests, and whether the override stays.
```

H-11 Accept or reject the remaining dependency risk

Who: Security reviewer **Time:** 15 min **Needs:** R-14

- Read the updated `dependency-advisories.md`. For each advisory still open (likely `braces`), record a decision: “accepted until <date> because <reachability evidence>”, or “must replace before <release>”.
- Set a calendar reminder to re-check monthly. An accepted risk is still an open advisory; it isn't fixed.

STAGE 4 · AUDIT P1 · R7, R8

Governed vertical slices

One real, end-to-end case each. These need people who are independent of the build team.

H-12 Appoint an independent verifier for XVL/VTE

Who: Founder / product owner **Time:** 1–2 weeks **Needs:** nothing

1. Pick a person or firm with no part in building or selling ATANDA. Internal KLARITY/SOCRATES reviews don't count.
2. Agree in writing: which verification tracks they cover, what source material they get, what counts as pass, fail and inconclusive, and how they sign receipts.
3. Choose one real source (not synthetic) for the first case. Get the owner's consent to use it.
4. Give the Agent the verifier's name/organisation and how they will sign, for R-15.

R-15 XVL/VTE: source-bound receipts and a real fail path

Needs: H-12 **Closes:** R7 (code side)

Replit Agent prompt

```
Task R-15 (audit R7): make one independent XVL/VTE verification case possible end to end, with
evidence bound to the exact source.

1. Map the current verification tracks and what live readiness says is incomplete. List what is
missing for one complete track.
2. Each verification receipt must record: the source artifact's hash, the verifier's identity (from
H-12: <name/org>), the verdict (PASS / FAIL / INCONCLUSIVE), the date, and a signature or signed
upload the verifier controls. A receipt whose hash no longer matches the source is invalid.
3. FAIL and INCONCLUSIVE must be first-class: they are stored, shown to the owner, logged in
vte_verification_failures, and block promotion. Test both.
4. Synthetic receipts stay possible for dev tests but must be marked synthetic and can never satisfy
the gate. Add a test that a synthetic receipt is rejected by the gate.
5. Do not run the real case yourself and do not create a receipt on the verifier's behalf.

Report: the track map, changes, tests, and the exact steps the verifier will follow.
```

GATE H-13 Run the first independent XVL/VTE case

Who: Independent verifier, observed by QA **Needs:** R-15, H-02 (published) **Clears:** R7

1. The verifier signs in to production with their own account and runs the agreed track on the agreed real source.
2. Ideally include one deliberate failure (a known-bad variant) to prove FAIL is recorded and blocks promotion.
3. Record the receipt IDs, verdicts, serving revision and date. The verification authority, not engineering, marks R7 met.

H-14 Name the approval and publication authorities

Who: Founder **Needs:** H-08

- **IMPRIMA:** name the person who may approve a Final publication, and their deputy.
- **SHORT SPRINTS:** name the verifier authorities (they're stored in `sprint_verifier_authorities`, which only an admin can change) and who decides eligibility and release.
- Pick one qualified, consented real source for each pilot, and write down what acceptance and rejection look like.

R-16 Pilot readiness for IMPRIMA Final and SHORT SPRINTS

Needs: H-08, H-14 **Closes:** R8 (code side)

Replit Agent prompt

```
Task R-16 (audit R8): get one IMPRIMA Final case and one SHORT SPRINTS case ready for a real pilot.
Production database guards are installed and verified (H-07/H-08 evidence: <paste date and
revision>).

1. Walk both workflows in DEV with test users and write down every step: who acts, which table rows
change, which events are logged, and what the restricted role can and cannot do.
2. Tests (dev): Final publication creates an immutable version; editing it is refused; a correction
makes a new version. A rejected SHORT SPRINTS verdict is stored and recoverable (resubmission
creates a new run and leaves the rejected one intact). A cross-owner attempt is refused.
3. Add an admin-only "pilot evidence" export per workflow: the event trail, actor identities,
hashes, verdicts and timestamps for one run, so the authority can sign it off.
4. Keep internal KLARITY/SOCRATES results clearly labelled "internal, not independent".
5. Do not run the pilot in production and do not set verifier authorities yourself.

Report: the step map, tests and the export format.
```

GATE H-15 Run the two protected pilots

Who: Named authorities from H-14, observed by QA

Needs: R-16, H-13 for anything that needs independent verification **Clears:** R8

1. In production, run one IMPRIMA Final case from source to approved publication. Try to edit the Final afterwards and confirm it's refused.
2. Run one SHORT SPRINTS case including a rejection, then the recovery path.
3. Export the pilot evidence from R-16, sign it, and file it with the serving revision. The authorities decide whether the externally governed gates open.

STAGE 5 · AUDIT P1 · R9, R10

External handoff and controlled operation

One provider and one target first. A subscription or an export file isn't evidence; a real provider-side result is.

H-16 Set up one real provider for F10 → F11 (Slack walkthrough)

Who: Integration owner + Slack workspace admin **Time:** 30 min **Needs:** nothing

1. Choose one target: **Slack notifications** is the simplest (Slack agent hosting needs a working agent endpoint).
2. Go to api.slack.com/apps → **Create New App** → **From scratch**. Name it “ATANDA Release”, pick your workspace.
3. **OAuth & Permissions → Bot Token Scopes:** add only what the Agent lists in R-17 (for posting to a channel this is usually `chat:write`). Least privilege.
4. **Install to Workspace**, approve. Keep the Bot User OAuth Token (`xoxb-…`) secret. Enter it only through the app's F10 provider connection screen, never in chat or files.
5. Find the IDs the app requires. In Slack on the web, open the channel; the URL is `app.slack.com/client/T…/C…` . `T…` is the workspace ID and `C…` is the channel ID. Invite the app to that channel (`/invite @ATANDA Release`).
6. Use a private test channel for the first run.

R-17 F10 → F11: artifact-bound handoff with a provider receipt

Needs: R-00 (H-16 for the live run) **Closes:** R9 (code side); tasks #335, #336

Replit Agent prompt

```
Task R-17 (audit R9; tasks #335, #336): make one F10 → F11 handoff to Slack (target
SLACK_NOTIFICATIONS) produce real, checkable evidence. Code lives in artifacts/api-
server/src/lib/f10-provider.ts, f10-export.ts and f10-collaboration.

1. List the exact Slack OAuth scopes the code needs for SLACK_NOTIFICATIONS and tell me, so the
human can grant only those (step H-16).
2. On dispatch, keep the provider's own response as the receipt: Slack's channel ID and message ts
from chat.postMessage, plus our artifact hash and the frozen authorization policy hash. Never store
the token in the receipt.
3. A delivery only counts as DELIVERED when the provider response confirms it. Timeout or unknown
means UNCERTAIN: shown to operators, not retried automatically.
4. Make clear in the UI and the data that a LangChain export file or LangSmith dataset upload is an
EXPORT, not a hosted execution (#335). Hosting receipts (#336) must come from the hosting provider,
bound to the artifact hash.
5. Tests with a mocked Slack API: success, revoked authorization (rechecked right before dispatch),
wrong channel, timeout.
6. Do not post to any real Slack workspace.

Report: scopes needed, receipt format, tests.
```

GATE H-17

Run the first real handoff

Who: Integration owner **Needs:** R-17 published, H-16 **Clears:** R9 for Slack notifications only

1. In the live app, connect the Slack provider with the token from H-16, workspace ID and channel ID.
2. Hand off one qualified artifact. Check the message appears in the test channel.
3. In the app, open the receipt and confirm it shows the Slack channel and message timestamp and the artifact hash.
4. Revoke the app in Slack (**Manage apps → Remove**) and try again. The app must refuse. Reinstall afterwards if you continue.
5. Record the evidence. Each additional provider or target needs its own run; there's no blanket switch.

R-18

F12: health hold, staged promotion and rollback

Needs: R-17 **Closes:** R10 (code side)

Replit Agent prompt

```
Task R-18 (audit R10): make F12 controlled operation work against one real target type, with a
proven rollback.

1. Map what F12 does today and what live readiness says is incomplete.
2. Implement for ONE target type (I will choose the target in H-18; build it against the replit-
deployments provider unless I say otherwise):
   - health check after each rollout step, with a timeout;
   - a failed or missing health result HOLDS the rollout (nothing gets promoted);
   - staged promotion (for example 10% then 100%, or staging then production), each stage recorded;
   - rollback to the previous known-good revision, recorded with who triggered it and the
before/after revision;
   - incident record when a hold or rollback happens, with an alert to the owner.
3. Tests with a fake target: healthy path, failing health (must hold), rollback restores the
previous revision, and a crash mid-rollout leaves a clear held state.
4. Don't touch any real target.

Report: state diagram, tests, and the exact steps for the rehearsal (H-18).
```

GATE H-18

Rehearse a failed release and a rollback on a real target

Who: Operations + release approver **Needs:** R-18 published **Clears:** R10

1. Pick a low-risk real target you own (a staging deployment, not the main site).
2. Release a known-good version through F12. Confirm health passes and it's promoted.
3. Release a deliberately broken version (for example, its health endpoint returns 500). Confirm F12 holds and doesn't promote.
4. Trigger rollback. Confirm the target serves the previous version again (check its revision header or version page).
5. Record each step's time, revision and result. The release approver signs off before wider use.

STAGE 6 · AUDIT R11

SPHINX public marketplace

Stays closed (`publicOpen: false`) until rights, withdrawal and sync work across both systems.

R-19

Marketplace contract: consent, withdrawal and sync

Needs: R-17 **Closes:** R11 (code side)

Replit Agent prompt

```
Task R-19 (audit R11): build the SPHINX source-to-marketplace contract while keeping the public
catalog CLOSED.

1. An Exemplar can be listed only with: recorded owner consent (opt-in, with date and terms
version), a qualification result, and a rights check. Test that each missing item blocks listing.
2. Metadata freshness: if the source changes after listing, the listing is flagged stale and hidden
until re-qualified.
3. Withdrawal: the owner can de-list at any time; the withdrawal is sent to the receiving site
(SPHINX_BASE_URL) and we keep the receiver's acknowledgement. Without an acknowledgement, show
"withdrawal pending" and retry with backoff.
4. Reconciliation job: compare our listings with the receiver's catalog and report differences both
ways.
5. GET /api/sphinx/catalog must keep returning publicOpen: false and items: [] until an admin flips
an explicit launch flag. Add a test that it stays closed by default.
6. Use a fake receiver in tests. Do not contact the real SPHINX site.

Report: contract, tests, and what the receiving site must implement.
```

GATE H-19

Decide whether to open the marketplace

Who: Marketplace owner + legal + product **Needs:** R-19 published **Clears:** R11

1. Legal: approve the listing terms, consent wording and withdrawal promise.
2. Confirm the receiving site implements withdrawal and acknowledgement (from R-19's report).
3. Run one real Exemplar through: list it, check it on the receiving site, withdraw it, and confirm it's gone on both.
4. Only then does an admin flip the launch flag. Record who decided, and when.

STAGE 7 · CLASS B, DEFERRED

Azure Key Vault

Agreed deferral. It doesn't block the core MVP. Do it after the earlier stages, without weakening current secret handling.

H-20

Provision the vault

Who: Security + platform **Time:** 1 hour **Needs:** an Azure subscription

1. In the Azure portal: **Create a resource → Key Vault**. Choose region, standard tier, **Azure role-based access control**, and turn on soft delete and purge protection.
2. **Microsoft Entra ID → App registrations → New registration** “atanda-api”. Under **Certificates & secrets**, create a client secret (or use federated credentials if your host supports them).
3. In the vault's **Access control (IAM)**, give that app the **Key Vault Secrets User** role (read only). Give humans who rotate secrets **Key Vault Secrets Officer**.
4. Put the tenant ID, client ID and client secret in Replit Secrets as `AZURE_TENANT_ID`, `AZURE_CLIENT_ID`, `AZURE_CLIENT_SECRET`, and the vault URL as `AZURE_KEY_VAULT_URL`.
5. Decide which secrets move first (suggest: F10 provider authorization keys and Stripe webhook secret) and the rotation schedule.

R-20 Read secrets from Key Vault with rotation

Needs: H-20 **Closes:** Class B

Replit Agent prompt

```
Task R-20 (Class B: Azure Key Vault). Azure credentials are in secrets AZURE_TENANT_ID,
AZURE_CLIENT_ID, AZURE_CLIENT_SECRET, AZURE_KEY_VAULT_URL.

1. Add a secret provider that reads from Key Vault (@azure/identity + @azure/keyvault-secrets) with
a short in-memory cache, and falls back to the current environment variable ONLY when
AZURE_KEY_VAULT_URL is unset. If the vault is configured but unreachable, fail closed for the
affected feature with a clear operator message. Never log secret values.
2. Move first: F10_PROVIDER_AUTHORIZATION_KEY / _VERSION (keep the existing current/previous
rotation rules in replit.md) and STRIPE_WEBHOOK_SECRET.
3. Rotation: support current + previous versions so a rotation needs no downtime, following the
existing F10 key rotation procedure.
4. Tests with a mocked vault: read, cache expiry, rotation to a new version, vault down.
5. Keep #341 (Azure ethical safeguards) in scope: no mandatory governance check may be removed when
adapting artifacts to Azure.
6. Write docs/upgrade-program/key-vault-rotation.md with the human rotation steps.

Report: changes, tests, rotation doc.
```

REFERENCE

Evidence record template

Use one per gate or fix, as the audit asks. Synthetic receipts, configuration presence, passing component tests and a successful deployment don't count as proof of real operation.

Template

```
Capability / fix:           (precisely bounded behaviour)
Responsible owner:          (person or team)
Implementation state:       MISSING | PARTIAL | IMPLEMENTED
Development verification:   (test type, result, date; say if mocked or synthetic)
Live activity:              ACTIVE | INACTIVE | UNVERIFIED
Production evidence:        (what was observed, by whom, date and time UTC)
Serving revision:           (X-Serving-Revision value at the time)
Blocking dependency:        (missing authority, implementation, configuration or evidence)
Exit condition:             (observable acceptance criteria)
Decision:                   (who decided, what, date)
```

Who owns which gate

GATE	TODAY	OPENED BY	STEPS
Published core	Active, revision unknown	Release engineering	R-02, H-02

GATE	TODAY	OPENED BY	STEPS
Protected IMPRIMA / SHORT SPRINTS database guards	Closed	Administrator + security reviewer	R-01, R-03, H-03 to H-08
IMPRIMA / SHORT SPRINTS external release	Conditional	Named authorities	H-14, R-16, H-15
Independent XVL/VTE	Incomplete	Verification authority	H-12, R-15, H-13
F10/F11	Conditional per provider	Integration owner	H-16, R-17, H-17
F12	Incomplete	Operations + release approver	R-18, H-18
SPHINX public	Closed	Marketplace + legal + product	R-19, H-19
Paid journey	Not attested	Business owner	R-13, H-10
Azure Key Vault	Deferred	Security + platform	H-20, R-20
Subscriber-wide database isolation	Separate pending scope	Security architecture	Not in this runbook; needs its own design review